Lecture Notes on Advanced Algorithms and Complexity

Asymptotic Analysis and Time Complexity

Author: ***Abdelkrim Mebarki***
Date : ***September 28, 2026***
Master's Program in Computer Science – All Options
Department of Computer Science – Faculty of Mathematics and Computer Science
University of Science and Technology of Oran – Mohamed Boudiaf (USTO-MB)

1. Course Overview

This lecture develops a systematic method for analyzing an algorithm's running time. The central goal is to move from an informal statement such as "this algorithm is fast" to a mathematical description of how computational cost grows as the input size increases.

The lecture focuses on time complexity rather than machine-dependent execution time. Students identify elementary operations, estimate their number of executions, express computational cost as a function of input size, and describe its asymptotic growth using O, Ω, and Θ notation.

Learning objectives

→ Identify elementary operations in simple algorithms.
→ Define the input-size parameter **n** appropriately.
→ Construct a computational-cost function from an algorithm.
→ Explain and distinguish **O**, **Ω**, and **Θ**.
→ Recognize common complexity classes.
→ Compare functions by asymptotic growth.
→ Relate increasing input size to computational cost.
→ Analyze simple iterative and nested-loop algorithms.

Topics covered

- The notion of an elementary operation
- Comparisons, assignments, arithmetic operations, and data accesses
- Measuring computational cost
- Big O and its definition and interpretation
- Omega and Theta
- Meaning and comparison of O, Ω, and Θ
- Asymptotic complexity
- **O(1), O(log n), O(n), O(n log n), O(n^2), O(n^3), O(2^n), O(n!)**
- Asymptotic comparison and growth rates
- Relationship between input size and computational cost

Teaching emphasis: *Students should not merely memorize complexity labels. They should justify a class by identifying the repeated work and explaining how that work grows with n.*

Available on zenodo – DOI : 10.5281/zenodo.23044459

2. The Notion of an Elementary Operation

An **elementary operation** is a basic computational action that can be treated as taking approximately constant time in the algorithmic model used for analysis. Typical examples are comparisons, assignments, basic arithmetic operations, and data accesses.

"**Constant time**" is a modeling assumption. It does not claim that every operation takes exactly the same number of processor cycles on every machine. Rather, it allows us to focus on how the number of operations changes as **n** changes.

2.1 Why count operations instead of seconds?

Wall-clock execution time depends on the processor, programming language, compiler, operating system, memory hierarchy, implementation details, and current system load. **Operation counting** gives a machine-independent way to reason about scalability.

***Key idea**: For asymptotic analysis, the exact cost of one elementary operation is less important than how many such operations are performed as a function of **n**.*

2.2 Typical elementary operations

Operation	Example	What is counted
Comparison	if (a[i] < x)	Evaluation of the comparison
Assignment	x = a[i]	Assignment of a value
Arithmetic	x = x + 1	Addition, subtraction, multiplication, etc.
Data access	a[i]	Access to a data item
Increment/decrement	i++	Counter update
Logical operation	p && q	Logical evaluation, according to the chosen model

2.3 Example

```
sum = 0
for i = 1 to n:
    sum = sum + a[i]
```

Each iteration accesses *a[i]*, performs an addition, and assigns the result to *sum*. Loop control also contributes comparisons and updates. A detailed count depends on the counting convention, but every contribution is proportional to n. Therefore the total cost is $\Theta(n)$.

This illustrates the distinction between exact operation counting and asymptotic analysis. An exact count might have the form $an + b$, whereas the asymptotic description is $\Theta(n)$.

3. Measuring the Computational Cost

3.1 Input size

The input size n is the parameter used to express how large an input is. Its meaning depends on the problem.

- For an array, n may be the number of elements.
- For a string, n may be the number of characters.
- For a graph, size may involve both vertices and edges.
- For an integer, the relevant size is often its number of bits rather than its numerical value.

A sound analysis therefore starts by explicitly defining n.

3.2 From operations to a cost function

```
for i = 1 to n:
    x = x + 1
```

The body executes n times. If the addition and assignment are counted separately, it contributes $2n$ elementary operations, plus loop-control work. The cost has the form $T(n) = an + b$ for constants a and b, so $T(n) = \Theta(n)$.

3.3 Exact cost versus asymptotic cost

Level	Example	Purpose
Exact count	3n + 2	Detailed operation accounting
Upper bound	O(n)	Asymptotic upper bound
Lower bound	Ω(n)	Asymptotic lower bound
Tight bound	Θ(n)	Asymptotic order of growth

4. Big O Notation : O

4.1 Formal definition

A function $f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for every $n \geq n_0$.

The constants do not depend on n. The definition permits us to ignore constant-factor differences and behavior for sufficiently small inputs.

Interpretation: *O(g(n)) says that, beyond some input size, f(n) does not grow faster than a constant multiple of g(n).*

4.2 Example: proving O(n)

Let $f(n) = 3n + 5$. For $n \geq 1$, $5 \leq 5n$. Hence $3n + 5 \leq 8n$. Choosing $c = 8$ and $n_0 = 1$ proves that $f(n)$ is $O(n)$.

4.3 What Big O does not mean

- It does not mean the algorithm performs exactly $g(n)$ operations.
- It does not mean running time is numerically equal to $g(n)$.
- It does not precisely describe small inputs.
- It is an asymptotic upper bound.

5. Omega Notation : Ω

A function $f(n)$ is $\Omega(g(n))$ if there exist positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for every $n \geq n_0$.

For $f(n) = 3n + 5$, we have $3n + 5 \geq 3n$ for $n \geq 1$. Therefore $f(n)$ is $\Omega(n)$.

Interpretation: *Ω(g(n)) states that f(n) grows at least as fast as a constant multiple of g(n), for sufficiently large n.*

6. Theta Notation : Θ

A function $f(n)$ is $\Theta(g(n))$ if there exist positive constants c_1, c_2, and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

Theta therefore establishes both an asymptotic lower and upper bound.

For $f(n) = 3n + 5$ and $n \geq 1$, we have $3n \leq 3n + 5 \leq 8n$. Thus $f(n) = \Theta(n)$.

6.2 Comparison

Notation	Meaning	Question answered
O(g(n))	Asymptotic upper bound	How fast can f grow at most?
Ω(g(n))	Asymptotic lower bound	How slowly can f grow at least?
Θ(g(n))	Tight asymptotic bound	What is the asymptotic order of growth?

Important: *If $f(n) = \Theta(g(n))$, then $f(n)$ is both $O(g(n))$ and $\Omega(g(n))$.*

7. Asymptotic Complexity

Asymptotic complexity characterizes computational growth for large input sizes. It ignores constant factors and lower-order terms because the dominant term determines the long-run growth.

7.1 Simplifying a cost function

For $T(n) = 4n^2 + 7n + 20$, the quadratic term dominates as n becomes large. Therefore $T(n) = \Theta(n^2)$. Similarly, $12n + 50$ is $\Theta(n)$, and $2n^3 + 4n^2 + n$ is $\Theta(n^3)$.

7.2 Why constants are ignored

Consider $T_1(n) = 100n$ and $T_2(n) = n^2$. For some small n, the first may have a larger numerical cost. Nevertheless, n^2 eventually grows beyond any constant multiple of n. Asymptotic analysis is concerned with this long-run growth behavior.

Teaching point: *Asymptotic analysis is primarily about scalability, not exact elapsed seconds on one particular computer.*

8. Common Complexity Classes

8.1 O(1) [Constant time]

The amount of work remains bounded independently of n.

```
x = A[5]
return x
```

Assuming direct array access, the operation count does not increase when the array becomes larger. The complexity is $\Theta(1)$.

8.2 O(log n) [Logarithmic time]

A logarithmic algorithm repeatedly reduces the remaining problem size by a constant factor. Binary search is the standard example.

```
while low <= high:
    mid = (low + high) // 2
    if A[mid] == x:
        return mid
    elif A[mid] < x:
        low = mid + 1
    else:
        high = mid - 1
```

After k halvings, the remaining interval is approximately $n/2^k$. Stopping when this becomes at most 1 gives $k = \Theta(\log n)$.

8.3 O(n) [Linear time]

```
for i = 1 to n:
    process(A[i])
```

The body executes n times and performs constant work, so the complexity is $\Theta(n)$.

8.4 O(n log n) [Linearithmic time

This often arises when there are logarithmically many levels, with $\Theta(n)$ total work performed at each level. A standard example is mergesort. The mental model is n work per level multiplied by $\log n$ levels.

8.5 O(n^2) [Quadratic time]

```
for i = 1 to n:
    for j = 1 to n:
        process(i, j)
```

The inner body executes n times for each of n outer iterations, giving n^2 executions. The complexity is $\Theta(n^2)$.

8.6 O(n^3) [Cubic time]

```
for i = 1 to n:
    for j = 1 to n:
        for k = 1 to n:
            process(i, j, k)
```

Three independent nested loops contribute three factors of n, giving $\Theta(n^3)$.

8.7 O(2^n) [Exponential time]

Exponential growth occurs when the amount of work grows roughly by a constant factor for each additional input element. Naive recursive Fibonacci is a standard introductory example.

```
fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
```

The recursive call tree expands rapidly. In elementary analysis this algorithm is commonly characterized with an exponential upper bound such as $O(2^n)$; more precise bounds can also be derived.

8.8 O(n!) [Factorial time]

Factorial growth appears when an algorithm enumerates all permutations of n distinct elements. There are $n!$ such permutations, so merely producing all of them already requires factorial-scale output.

8.9 Summary

Class	Typical pattern	Growth intuition
O(1)	Direct access	Does not grow with n
O(log n)	Repeated halving	Very slow growth
O(n)	Single pass	Proportional to n
O(n log n)	Linear work over logarithmic levels	Moderate growth
O(n^2)	Two nested loops	Quadratic growth
O(n^3)	Three nested loops	Cubic growth
O(2^n)	Branching recursion	Rapid exponential growth
O(n!)	Permutation enumeration	Extremely rapid growth

9. Asymptotic Comparison of Functions

9.1 Dominant terms

Function	Dominant term	Class
$7n + 20$	n	$\Theta(n)$
$4n^2 + 3n + 8$	n^2	$\Theta(n^2)$
$2n^3 + 5n^2 + n$	n^3	$\Theta(n^3)$
$5 \log n + 12$	$\log n$	$\Theta(\log n)$
$3n \log n + 2n$	$n \log n$	$\Theta(n \log n)$

9.2 Growth-rate ordering

For the standard classes in this course, the usual asymptotic ordering is:

$$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$$

The symbol $\prec$ here means "grows asymptotically more slowly than." This is a statement about sufficiently large n, not necessarily every small input.

9.3 Example: n log n versus n^2

$(n \log n)/n^2 = (\log n)/n$, and $\log n / n$ tends to 0 as n grows. Thus $n \log n$ grows more slowly than n^2.

10. Relationship Between Input Size and Computational Cost

Different complexity classes react very differently to increasing input size. This is the practical reason complexity analysis matters.

Change in n	O(1)	O(log n)	O(n)	$O(n^2)$	$O(2^n)$
n doubles	approximately unchanged	small increase	approximately 2×	approximately 4×	approximately squared
n increases by 1	approximately unchanged	small increase	linear-scale extra work	quadratic-scale extra work	roughly doubles

These are qualitative growth relationships; exact numerical factors depend on the particular function.

10.1 Scalability

An algorithm that is practical for $n = 100$ may become impractical for $n = 100{,}000$ if its cost is quadratic or worse. Linear and logarithmic algorithms generally scale more gently. Complexity analysis therefore helps designers anticipate whether an approach remains viable as datasets grow.

11. Worked Complexity-Analysis Examples

Example 1 — Single loop

```
count = 0
for i = 1 to n:
    count = count + 1
```

The loop executes n times and the body has constant cost. Therefore $T(n) = an + b$ for constants a and b, giving $\Theta(n)$.

Example 2 — Two consecutive loops

```
for i = 1 to n:
    process(A[i])

for j = 1 to n:
    process(B[j])
```

The two loops have costs $\Theta(n)$ and $\Theta(n)$. Consecutive costs add: $\Theta(n) + \Theta(n) = \Theta(n)$. They are not multiplied because the second loop does not execute inside the first.

Example 3 — Nested loops

```
for i = 1 to n:
    for j = 1 to n:
        process(i, j)
```

The inner statement executes n times for each of n outer iterations, giving n^2 executions. Hence the complexity is $\Theta(n^2)$.

Example 4 — Triangular nested loop

```
for i = 1 to n:
    for j = 1 to i:
        process(i, j)
```

The inner loop executes i times during outer iteration i. Thus the total is $1 + 2 + \ldots + n = n(n + 1)/2 = n^2/2 + n/2$. Therefore the complexity is $\Theta(n^2)$.

Example 5 — Repeated halving

```
i = n
while i > 1:
    i = floor(i / 2)
```

After k iterations, i is approximately $n/2^k$. The loop stops when $n/2^k \leq 1$, so k is proportional to $\log_2 n$. Therefore the complexity is $\Theta(\log n)$.

Example 6 — Mixed linear and logarithmic loops

```
for i = 1 to n:
    j = 1
    while j <= n:
        process(i, j)
        j = 2*j
```

The outer loop runs n times. The inner variable doubles, so the inner loop runs $\Theta(\log n)$ times.

The total is $\Theta(n \log n)$.

12. Common Student Mistakes

- Confusing O(n) with exactly n operations.
- Multiplying consecutive loop costs instead of adding them.
- Forgetting that nested loops often multiply iteration counts.
- Assuming every loop is O(n).
- Keeping lower-order terms when deriving asymptotic complexity.
- Treating O, Ω, and Θ as interchangeable.
- Failing to define what n represents.
- Using measured seconds as the definition of asymptotic complexity.
- Assuming a lower asymptotic class is always numerically faster for every small input.
- Ignoring repeated operations inside loops.

13. A Practical Procedure for Analyzing an Algorithm

1. Define the input-size parameter n.
2. Identify the basic operation or operations whose repetitions dominate the cost.
3. Determine how many times the relevant operation is executed.
4. Write the cost function T(n), when useful.

5. Keep the dominant asymptotic term.
6. State the resulting O, Ω, or Θ bound.
7. Check the result against the algorithmic structure: single loop, nested loops, halving, branching, and so on.

Recommended habit: *Do not start by guessing the complexity class. First identify and count the repeated work; then derive the asymptotic class.*

17. Lecture Summary

Time-complexity analysis provides a mathematical way to reason about how an algorithm scales with input size. The analysis begins by identifying elementary operations and determining how often they occur. The resulting cost function is then simplified asymptotically.

- Elementary operations provide a basic unit for computational-cost analysis.
- Input size n must be defined appropriately.
- Big O provides an asymptotic upper bound.
- Omega provides an asymptotic lower bound.
- Theta provides a tight asymptotic bound.
- Dominant terms determine asymptotic growth.
- Common classes range from constant and logarithmic to linear, linearithmic, polynomial, exponential, and factorial growth.
- The main purpose is to understand scalability as input size increases.

Take-home message: *A good complexity analysis explains not only which class an algorithm belongs to, but why its computational cost grows in that way.*

18. Further Reading

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: The MIT Press, 2022. ISBN: 978-0-262-04630-5.

[2] J. Kleinberg and É. Tardos, Algorithm Design, 1st ed. Boston, MA, USA: Addison-Wesley, 2005. ISBN: 978-0-321-29535-4.

[3] S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani, Algorithms, 1st ed. New York, NY, USA: McGraw-Hill, 2006. ISBN: 978-0-07-352340-8.

[4] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011. ISBN: 978-0-321-57351-3.